Modern Cryptography

Laboratory Programs and Captured Outputs

Name: Mohnish Condoor
Registration No.: 25011103033
Semester: III
Language: C++17

Index

Ex.	Date	Title of the Experiment	Page
1	13.07.2026	Introduction to Modern Cryptography Algorithms	2
2	20.07.2026	Caesar, Vigenere and Hill Ciphers	3
3	27.07.2026	One-Time Pad (OTP)	7
4	03.08.2026	Feistel Network	8
5	10.08.2026	Data Encryption Standard (DES)	9
6	17.08.2026	Triple DES (3DES)	12
7	24.08.2026	Advanced Encryption Standard (AES)	15
8	05.10.2026	Miller-Rabin Primality Test	18
9	12.10.2026	RSA Encryption	20

About this record. The supplied index lists experiment titles; the question statements below are implementation tasks formulated from those titles. Experiment 2 contains three separate programs. Every output was captured from a compiled C++ program in a terminal with input echo enabled.

Compilation. Each listing is a complete, standalone program. For example:

```
mkdir -p build
g++ -std=c++17 -O2 -Wall -Wextra aes.cpp -o build/aes
./build/aes
```

GCC or Clang is needed for the 128-bit intermediate arithmetic in Miller-Rabin and RSA. No external cryptography library is required by the programs.

Scope. These are laboratory demonstrations. DES and 3DES are legacy algorithms; block cipher examples process one block without a mode, padding or authentication. RSA uses small textbook parameters. The sample OTP key and Feistel round function are for demonstration.

EXPERIMENT 1 **Date: 13.07.2026**

Introduction to Modern Cryptography Algorithms

Question

Write a C++ program to compute the greatest common divisor, modular exponentiation and modular inverse for given nonnegative base and exponent, and a modulus greater than one. Report when the inverse does not exist.

The modulus is limited to 1,000,000,000 so intermediate products fit in signed 64-bit arithmetic.

C++ Program

`introduction.cpp`

```cpp
#include <iostream>
#include <numeric>
using namespace std;

long long modular_power(long long base, long long exponent,
                        long long modulus) {
  if (exponent == 0) return 1 % modulus;
  long long half = modular_power(base, exponent / 2, modulus);
  long long result = half * half % modulus;
  if (exponent % 2 == 1) result = result * base % modulus;
  return result;
}

long long extended_gcd(long long a, long long b,
                       long long &x, long long &y) {
  if (b == 0) {
    x = 1;
    y = 0;
    return a;
  }
  long long nextX, nextY;
  long long result = extended_gcd(b, a % b, nextX, nextY);
  x = nextY;
  y = nextX - (a / b) * nextY;
  return result;
}

int main() {
  long long base, exponent, modulus;
  cout << "Enter base, exponent and modulus: ";
  if (!(cin >> base >> exponent >> modulus) || base < 0 ||
      exponent < 0 || modulus < 2 || modulus > 1000000000) {
    cout << "Use base >= 0, exponent >= 0 and modulus 2..1000000000.\n";
    return 1;
  }
  base %= modulus;
  cout << "GCD: " << gcd(base, modulus) << endl;
  cout << "Modular power: "
       << modular_power(base, exponent, modulus) << endl;
  long long x, y;
  if (extended_gcd(base, modulus, x, y) == 1) {
    cout << "Modular inverse: " << (x % modulus + modulus) % modulus
         << endl;
  } else {
    cout << "Modular inverse does not exist.\n";
  }
  return 0;
}
```

Captured Output

```
$ ./build/introduction
Enter base, exponent and modulus: 7 128 26
GCD: 1
Modular power: 3
Modular inverse: 15
```

EXPERIMENT 2(a) **Date: 20.07.2026**

Caesar Cipher

Question

Write a C++ program to encrypt and decrypt a message using the Caesar cipher with a user-supplied integer shift. Preserve letter case, spaces and punctuation.

C++ Program

caesar.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

string caesar_cipher(string message, int shift) {
  shift = (shift % 26 + 26) % 26;
  for (char &letter : message) {
    if (letter >= 'A' && letter <= 'Z') {
      letter = 'A' + (letter - 'A' + shift) % 26;
    } else if (letter >= 'a' && letter <= 'z') {
      letter = 'a' + (letter - 'a' + shift) % 26;
    }
  }
  return message;
}

int main() {
  string message;
  int shift;
  cout << "Enter the message: ";
  getline(cin, message);
  cout << "Enter the shift: ";
  if (!(cin >> shift)) return 1;
  shift = (shift % 26 + 26) % 26;
  string encrypted = caesar_cipher(message, shift);
  cout << "Encrypted: " << encrypted << endl;
  cout << "Decrypted: " << caesar_cipher(encrypted, -shift) << endl;
  return 0;
}
```

Captured Output

```
$ ./build/caesar
Enter the message: Attack at Dawn!
Enter the shift: 3
Encrypted: Dwwdfn dw Gdzq!
Decrypted: Attack at Dawn!
```

EXPERIMENT 2(b) **Date: 20.07.2026**

Vigenere Cipher

Question

Write a C++ program to encrypt and decrypt a message using the Vigenere cipher and a nonempty alphabetic key. Preserve letter case and punctuation; advance the key only for letters.

C++ Program

vigenere.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

string vigenere_cipher(string message, string key, bool decrypt) {
  int position = 0;
  for (char &letter : message) {
    char start;
    if (letter >= 'A' && letter <= 'Z') start = 'A';
    else if (letter >= 'a' && letter <= 'z') start = 'a';
    else continue;
    int shift = key[position % key.size()] - 'A';
    if (decrypt) shift = -shift;
    letter = start + (letter - start + shift + 26) % 26;
    position++;
  }
  return message;
}

int main() {
  string message, key;
  cout << "Enter the message: ";
  getline(cin, message);
  cout << "Enter the key: ";
  getline(cin, key);
  if (key.empty()) {
    cout << "The key cannot be empty.\n";
    return 1;
  }
  for (char &letter : key) {
    if (letter >= 'a' && letter <= 'z') letter -= 'a' - 'A';
    if (letter < 'A' || letter > 'Z') {
      cout << "The key must contain only letters.\n";
      return 1;
    }
  }
  string encrypted = vigenere_cipher(message, key, false);
  cout << "Encrypted: " << encrypted << endl;
  cout << "Decrypted: " << vigenere_cipher(encrypted, key, true)
       << endl;
  return 0;
}
```

Captured Output

```
$ ./build/vigenere
Enter the message: ATTACK AT DAWN!
Enter the key: LEMON
Encrypted: LXFOPV EF RNHR!
Decrypted: ATTACK AT DAWN!
```

EXPERIMENT 2(c) **Date: 20.07.2026**

Hill Cipher

Question

Write a C++ program to encrypt and decrypt a message using the Hill cipher with an invertible 2 by 2 key matrix modulo 26. Use A = 0 through Z = 25 and column vectors.

Convert letters to uppercase, discard nonletters and append X if the letter count is odd. Remove only the padding added during this run.

C++ Program

hill.cpp

```cpp
#include <iostream>
#include <string>
#include <vector>
using namespace std;

int mod26(int number) {
  return (number % 26 + 26) % 26;
}

string hill_cipher(string message, vector<vector<int>> key) {
  string result;
  for (int i = 0; i < (int)message.size(); i += 2) {
    int first = message[i] - 'A';
    int second = message[i + 1] - 'A';
    result += 'A' + mod26(key[0][0] * first + key[0][1] * second);
    result += 'A' + mod26(key[1][0] * first + key[1][1] * second);
  }
  return result;
}
```

Experiment 2(c): Hill Cipher — C++ Program (continued)

```
int main() {
  string input, message;
  vector<vector<int>> key(2, vector<int>(2));
  cout << "Enter the message: ";
  getline(cin, input);
  for (char letter : input) {
    if (letter >= 'a' && letter <= 'z') letter -= 'a' - 'A';
    if (letter >= 'A' && letter <= 'Z') message += letter;
  }
  cout << "Enter the 2 x 2 key matrix (row-wise): ";
  for (auto &row : key) {
    for (int &value : row) {
      if (!(cin >> value)) return 1;
      value = mod26(value);
    }
  }
  int determinant = mod26(key[0][0] * key[1][1] -
                          key[0][1] * key[1][0]);
  int inverse = -1;
  for (int i = 1; i < 26; i++) {
    if (determinant * i % 26 == 1) inverse = i;
  }
  if (inverse == -1) {
    cout << "The key matrix is not invertible modulo 26.\n";
    return 1;
  }
  vector<vector<int>> inverseKey = {
      {mod26(inverse * key[1][1]), mod26(-inverse * key[0][1])},
      {mod26(-inverse * key[1][0]), mod26(inverse * key[0][0])}};
  bool padded = message.size() % 2 == 1;
  if (padded) message += 'X';
  string encrypted = hill_cipher(message, key);
  string decrypted = hill_cipher(encrypted, inverseKey);
  if (padded) decrypted.pop_back();
  cout << "Prepared message: " << message << endl;
  cout << "Encrypted: " << encrypted << endl;
  cout << "Decrypted: " << decrypted << endl;
  return 0;
}
```

Captured Output

```
$ ./build/hill
Enter the message: HELP
Enter the 2 x 2 key matrix (row-wise): 3 3 2 5
Prepared message: HELP
Encrypted: HIAT
Decrypted: HELP
```

EXPERIMENT 3 **Date: 27.07.2026**

One-Time Pad (OTP)

Question

Write a C++ program to encrypt and decrypt a message using byte-wise XOR with a supplied one-time key of exactly the same byte length. Read the key in hexadecimal and display the ciphertext in hexadecimal.

The sample key demonstrates XOR. A true OTP requires a uniformly random secret key that is used exactly once.

C++ Program

one-time-pad.cpp

```cpp
#include <iostream>
#include <iomanip>
#include <sstream>
#include <string>
using namespace std;

int hex_value(char digit) {
  if (digit >= '0' && digit <= '9') return digit - '0';
  if (digit >= 'a' && digit <= 'f') return digit - 'a' + 10;
  if (digit >= 'A' && digit <= 'F') return digit - 'A' + 10;
  return -1;
}

int main() {
  string message, keyHex, key, encrypted, decrypted;
  cout << "Enter the message: ";
  getline(cin, message);
  cout << "Enter the one-time key in hex (2 digits per byte): ";
  getline(cin, keyHex);
  if (message.empty() || keyHex.size() != 2 * message.size()) {
    cout << "Use a nonempty message and one key byte per message byte.\n";
    return 1;
  }
  for (int i = 0; i < (int)keyHex.size(); i += 2) {
    int high = hex_value(keyHex[i]);
    int low = hex_value(keyHex[i + 1]);
    if (high == -1 || low == -1) {
      cout << "Invalid hexadecimal key.\n";
      return 1;
    }
    key += char(high * 16 + low);
  }
  ostringstream output;
  output << hex << uppercase << setfill('0');
  for (int i = 0; i < (int)message.size(); i++) {
    unsigned char value = (unsigned char)message[i] ^
                          (unsigned char)key[i];
    encrypted += char(value);
    output << setw(2) << (int)value;
  }
  for (int i = 0; i < (int)encrypted.size(); i++) {
    decrypted += char((unsigned char)encrypted[i] ^
                      (unsigned char)key[i]);
  }
  cout << "Ciphertext (hex): " << output.str() << endl;
  cout << "Decrypted: " << decrypted << endl;
  return 0;
}
```

Captured Output

```
$ ./build/one-time-pad
Enter the message: HELLO
Enter the one-time key in hex (2 digits per byte): 1A2B3C4D5E
Ciphertext (hex): 526E700111
Decrypted: HELLO
```

EXPERIMENT 4 **Date: 03.08.2026**

Feistel Network

Question

Write a C++ program to encrypt and decrypt a 32-bit block using a four-round Feistel network, two 16-bit halves and four user-supplied 16-bit round keys. Reverse the key order for decryption.

Use $F(R,K) = \mathrm{ROTL}_{16}((R+K) \bmod 2^{16}, 3)$ and a final half swap. This is a teaching round function.

C++ Program

`feistel.cpp`

```cpp
#include <iostream>
#include <iomanip>
#include <vector>
using namespace std;

unsigned int round_function(unsigned int right, unsigned int key) {
  unsigned int value = (right + key) & 0xFFFF;
  return ((value << 3) | (value >> 13)) & 0xFFFF;
}

unsigned int feistel_cipher(unsigned int block,
                            vector<unsigned int> keys, bool decrypt) {
  unsigned int left = block >> 16;
  unsigned int right = block & 0xFFFF;
  for (int i = 0; i < (int)keys.size(); i++) {
    int index = decrypt ? (int)keys.size() - 1 - i : i;
    unsigned int next = left ^ round_function(right, keys[index]);
    left = right;
    right = next;
  }
  return (right << 16) | left;
}

int main() {
  unsigned long long block;
  vector<unsigned int> keys(4);
  cout << "Enter a 32-bit block in hex: ";
  if (!(cin >> hex >> block) || block > 0xFFFFFFFFULL) {
    cout << "Invalid 32-bit block.\n";
    return 1;
  }
  cout << "Enter four 16-bit round keys in hex: ";
  for (unsigned int &key : keys) {
    if (!(cin >> hex >> key) || key > 0xFFFF) {
      cout << "Invalid 16-bit round key.\n";
      return 1;
    }
  }
  unsigned int encrypted = feistel_cipher(block, keys, false);
  cout << hex << uppercase << setfill('0');
  cout << "Encrypted: " << setw(8) << encrypted << endl;
  cout << "Decrypted: " << setw(8)
       << feistel_cipher(encrypted, keys, true) << endl;
  return 0;
}
```

Captured Output

```
$ ./build/feistel
Enter a 32-bit block in hex: 12345678
Enter four 16-bit round keys in hex: 1111 2222 3333 4444
Encrypted: 2FA81B57
Decrypted: 12345678
```

EXPERIMENT 5 **Date: 10.08.2026**

Data Encryption Standard (DES)

Question

Write a C++ program to implement DES encryption and decryption of one 64-bit block using a 64-bit key representation. Include the initial and final permutations, key schedule, expansion, S-boxes and all 16 rounds.

Enter exactly 16 hexadecimal digits per block and key. DES uses 56 effective key bits; the eight parity positions are discarded by PC-1. No padding is added.

C++ Program

des.cpp

```
#include <iostream>
#include <iomanip>
#include <string>
#include <vector>
#include <cstdint>
using namespace std;

const int initialPermutation[64] = {
  58,50,42,34,26,18,10,2,60,52,44,36,28,20,12,4,
  62,54,46,38,30,22,14,6,64,56,48,40,32,24,16,8,
  57,49,41,33,25,17,9,1,59,51,43,35,27,19,11,3,
  61,53,45,37,29,21,13,5,63,55,47,39,31,23,15,7
};
const int finalPermutation[64] = {
  40,8,48,16,56,24,64,32,39,7,47,15,55,23,63,31,
  38,6,46,14,54,22,62,30,37,5,45,13,53,21,61,29,
  36,4,44,12,52,20,60,28,35,3,43,11,51,19,59,27,
  34,2,42,10,50,18,58,26,33,1,41,9,49,17,57,25
};
const int expansion[48] = {
  32,1,2,3,4,5,4,5,6,7,8,9,8,9,10,11,12,13,
  12,13,14,15,16,17,16,17,18,19,20,21,
  20,21,22,23,24,25,24,25,26,27,28,29,28,29,30,31,32,1
};
const int permutation[32] = {
  16,7,20,21,29,12,28,17,1,15,23,26,5,18,31,10,
  2,8,24,14,32,27,3,9,19,13,30,6,22,11,4,25
};
const int pc1[56] = {
  57,49,41,33,25,17,9,1,58,50,42,34,26,18,
  10,2,59,51,43,35,27,19,11,3,60,52,44,36,
  63,55,47,39,31,23,15,7,62,54,46,38,30,22,
  14,6,61,53,45,37,29,21,13,5,28,20,12,4
};
const int pc2[48] = {
  14,17,11,24,1,5,3,28,15,6,21,10,
  23,19,12,4,26,8,16,7,27,20,13,2,
  41,52,31,37,47,55,30,40,51,45,33,48,
  44,49,39,56,34,53,46,42,50,36,29,32
};
const int shifts[16] = {1,1,2,2,2,2,2,2,1,2,2,2,2,2,2,1};
const int sbox[8][4][16] = {
  {{14,4,13,1,2,15,11,8,3,10,6,12,5,9,0,7},
   {0,15,7,4,14,2,13,1,10,6,12,11,9,5,3,8},
   {4,1,14,8,13,6,2,11,15,12,9,7,3,10,5,0},
   {15,12,8,2,4,9,1,7,5,11,3,14,10,0,6,13}},
  {{15,1,8,14,6,11,3,4,9,7,2,13,12,0,5,10},
   {3,13,4,7,15,2,8,14,12,0,1,10,6,9,11,5},
   {0,14,7,11,10,4,13,1,5,8,12,6,9,3,2,15},
   {13,8,10,1,3,15,4,2,11,6,7,12,0,5,14,9}},
  {{10,0,9,14,6,3,15,5,1,13,12,7,11,4,2,8},
   {13,7,0,9,3,4,6,10,2,8,5,14,12,11,15,1},
   {13,6,4,9,8,15,3,0,11,1,2,12,5,10,14,7},
   {1,10,13,0,6,9,8,7,4,15,14,3,11,5,2,12}},
  {{7,13,14,3,0,6,9,10,1,2,8,5,11,12,4,15},
   {13,8,11,5,6,15,0,3,4,7,2,12,1,10,14,9},
   {10,6,9,0,12,11,7,13,15,1,3,14,5,2,8,4},
   {3,15,0,6,10,1,13,8,9,4,5,11,12,7,2,14}},
  {{2,12,4,1,7,10,11,6,8,5,3,15,13,0,14,9},
```

```
    {14,11,2,12,4,7,13,1,5,0,15,10,3,9,8,6},
    {4,2,1,11,10,13,7,8,15,9,12,5,6,3,0,14},
    {11,8,12,7,1,14,2,13,6,15,0,9,10,4,5,3}},
   {{12,1,10,15,9,2,6,8,0,13,3,4,14,7,5,11},
    {10,15,4,2,7,12,9,5,6,1,13,14,0,11,3,8},
    {9,14,15,5,2,8,12,3,7,0,4,10,1,13,11,6},
    {4,3,2,12,9,5,15,10,11,14,1,7,6,0,8,13}},
   {{4,11,2,14,15,0,8,13,3,12,9,7,5,10,6,1},
    {13,0,11,7,4,9,1,10,14,3,5,12,2,15,8,6},
    {1,4,11,13,12,3,7,14,10,15,6,8,0,5,9,2},
    {6,11,13,8,1,4,10,7,9,5,0,15,14,2,3,12}},
   {{13,2,8,4,6,15,11,1,10,9,3,14,5,0,12,7},
    {1,15,13,8,10,3,7,4,12,5,6,11,0,14,9,2},
    {7,11,4,1,9,12,14,2,0,6,10,13,15,3,5,8},
    {2,1,14,7,4,10,8,13,15,12,9,0,3,5,6,11}}
};

uint64_t permute(uint64_t input, const int table[],
                 int outputBits, int inputBits) {
  uint64_t result = 0;
  for (int i = 0; i < outputBits; i++) {
    result = (result << 1) | ((input >> (inputBits - table[i])) & 1);
  }
  return result;
}

vector<uint64_t> generate_keys(uint64_t key) {
  uint64_t reduced = permute(key, pc1, 56, 64);
  uint32_t left = reduced >> 28;
  uint32_t right = reduced & 0x0FFFFFFF;
  vector<uint64_t> keys;
  for (int shift : shifts) {
    left = ((left << shift) | (left >> (28 - shift))) & 0x0FFFFFFF;
    right = ((right << shift) | (right >> (28 - shift))) & 0x0FFFFFFF;
    keys.push_back(permute((uint64_t(left) << 28) | right, pc2, 48, 56));
  }
  return keys;
}

uint32_t round_function(uint32_t right, uint64_t key) {
  uint64_t expanded = permute(right, expansion, 48, 32) ^ key;
  uint32_t result = 0;
  for (int i = 0; i < 8; i++) {
    int value = (expanded >> (42 - 6 * i)) & 63;
    int row = ((value & 32) >> 4) | (value & 1);
    int column = (value >> 1) & 15;
    result = (result << 4) | sbox[i][row][column];
  }
  return permute(result, permutation, 32, 32);
}

uint64_t des_cipher(uint64_t block, uint64_t key, bool decrypt) {
  vector<uint64_t> keys = generate_keys(key);
  uint64_t initial = permute(block, initialPermutation, 64, 64);
  uint32_t left = initial >> 32;
  uint32_t right = initial & 0xFFFFFFFF;
  for (int i = 0; i < 16; i++) {
    uint32_t next = left ^ round_function(right, keys[decrypt ? 15 - i : i]);
    left = right;
    right = next;
  }
  return permute((uint64_t(right) << 32) | left, finalPermutation, 64, 64);
}

bool read_hex(string prompt, uint64_t &value) {
  string input;
  cout << prompt;
  if (!(cin >> input) || input.size() != 16) return false;
  value = 0;
  for (char digit : input) {
    int number;
    if (digit >= '0' && digit <= '9') number = digit - '0';
    else if (digit >= 'A' && digit <= 'F') number = digit - 'A' + 10;
    else if (digit >= 'a' && digit <= 'f') number = digit - 'a' + 10;
    else return false;
```

```
    value = (value << 4) | number;
  }
  return true;
}

int main() {
  uint64_t block, key;
  if (!read_hex("Enter the plaintext (16 hex digits): ", block) ||
      !read_hex("Enter the key (16 hex digits): ", key)) {
    cout << "Each input must contain exactly 16 hex digits.\n";
    return 1;
  }
  uint64_t encrypted = des_cipher(block, key, false);
  cout << hex << uppercase << setfill('0');
  cout << "Encrypted: " << setw(16) << encrypted << endl;
  cout << "Decrypted: " << setw(16)
       << des_cipher(encrypted, key, true) << endl;
  return 0;
}
```

Captured Output

```
$ ./build/des
Enter the plaintext (16 hex digits): 0123456789ABCDEF
Enter the key (16 hex digits): 133457799BBCDFF1
Encrypted: 85E813540F0AB405
Decrypted: 0123456789ABCDEF
```

EXPERIMENT 6 **Date: 17.08.2026**

Triple DES (3DES)

Question

Write a C++ program to encrypt and decrypt one 64-bit block using three-key Triple DES in encrypt-decrypt-encrypt (EDE) order. Implement DES directly and reverse the operations for decryption.

Encryption: $C = E_{K_3}(D_{K_2}(E_{K_1}(P)))$. Enter 16 hexadecimal digits for the block and each of the three keys. No padding is added.

C++ Program

`triple-des.cpp`

```cpp
#include <iostream>
#include <iomanip>
#include <string>
#include <vector>
#include <cstdint>
using namespace std;

const int initialPermutation[64] = {
  58,50,42,34,26,18,10,2,60,52,44,36,28,20,12,4,
  62,54,46,38,30,22,14,6,64,56,48,40,32,24,16,8,
  57,49,41,33,25,17,9,1,59,51,43,35,27,19,11,3,
  61,53,45,37,29,21,13,5,63,55,47,39,31,23,15,7
};
const int finalPermutation[64] = {
  40,8,48,16,56,24,64,32,39,7,47,15,55,23,63,31,
  38,6,46,14,54,22,62,30,37,5,45,13,53,21,61,29,
  36,4,44,12,52,20,60,28,35,3,43,11,51,19,59,27,
  34,2,42,10,50,18,58,26,33,1,41,9,49,17,57,25
};
const int expansion[48] = {
  32,1,2,3,4,5,4,5,6,7,8,9,8,9,10,11,12,13,
  12,13,14,15,16,17,16,17,18,19,20,21,
  20,21,22,23,24,25,24,25,26,27,28,29,28,29,30,31,32,1
};
const int permutation[32] = {
  16,7,20,21,29,12,28,17,1,15,23,26,5,18,31,10,
  2,8,24,14,32,27,3,9,19,13,30,6,22,11,4,25
};
const int pc1[56] = {
  57,49,41,33,25,17,9,1,58,50,42,34,26,18,
  10,2,59,51,43,35,27,19,11,3,60,52,44,36,
  63,55,47,39,31,23,15,7,62,54,46,38,30,22,
  14,6,61,53,45,37,29,21,13,5,28,20,12,4
};
const int pc2[48] = {
  14,17,11,24,1,5,3,28,15,6,21,10,
  23,19,12,4,26,8,16,7,27,20,13,2,
  41,52,31,37,47,55,30,40,51,45,33,48,
  44,49,39,56,34,53,46,42,50,36,29,32
};
const int shifts[16] = {1,1,2,2,2,2,2,2,1,2,2,2,2,2,2,1};
const int sbox[8][4][16] = {
  {{14,4,13,1,2,15,11,8,3,10,6,12,5,9,0,7},
   {0,15,7,4,14,2,13,1,10,6,12,11,9,5,3,8},
   {4,1,14,8,13,6,2,11,15,12,9,7,3,10,5,0},
   {15,12,8,2,4,9,1,7,5,11,3,14,10,0,6,13}},
  {{15,1,8,14,6,11,3,4,9,7,2,13,12,0,5,10},
   {3,13,4,7,15,2,8,14,12,0,1,10,6,9,11,5},
   {0,14,7,11,10,4,13,1,5,8,12,6,9,3,2,15},
   {13,8,10,1,3,15,4,2,11,6,7,12,0,5,14,9}},
  {{10,0,9,14,6,3,15,5,1,13,12,7,11,4,2,8},
   {13,7,0,9,3,4,6,10,2,8,5,14,12,11,15,1},
   {13,6,4,9,8,15,3,0,11,1,2,12,5,10,14,7},
   {1,10,13,0,6,9,8,7,4,15,14,3,11,5,2,12}},
  {{7,13,14,3,0,6,9,10,1,2,8,5,11,12,4,15},
   {13,8,11,5,6,15,0,3,4,7,2,12,1,10,14,9},
   {10,6,9,0,12,11,7,13,15,1,3,14,5,2,8,4},
   {3,15,0,6,10,1,13,8,9,4,5,11,12,7,2,14}},
  {{2,12,4,1,7,10,11,6,8,5,3,15,13,0,14,9},
```

```
    {14,11,2,12,4,7,13,1,5,0,15,10,3,9,8,6},
    {4,2,1,11,10,13,7,8,15,9,12,5,6,3,0,14},
    {11,8,12,7,1,14,2,13,6,15,0,9,10,4,5,3}},
   {{12,1,10,15,9,2,6,8,0,13,3,4,14,7,5,11},
    {10,15,4,2,7,12,9,5,6,1,13,14,0,11,3,8},
    {9,14,15,5,2,8,12,3,7,0,4,10,1,13,11,6},
    {4,3,2,12,9,5,15,10,11,14,1,7,6,0,8,13}},
   {{4,11,2,14,15,0,8,13,3,12,9,7,5,10,6,1},
    {13,0,11,7,4,9,1,10,14,3,5,12,2,15,8,6},
    {1,4,11,13,12,3,7,14,10,15,6,8,0,5,9,2},
    {6,11,13,8,1,4,10,7,9,5,0,15,14,2,3,12}},
   {{13,2,8,4,6,15,11,1,10,9,3,14,5,0,12,7},
    {1,15,13,8,10,3,7,4,12,5,6,11,0,14,9,2},
    {7,11,4,1,9,12,14,2,0,6,10,13,15,3,5,8},
    {2,1,14,7,4,10,8,13,15,12,9,0,3,5,6,11}}
};

uint64_t permute(uint64_t input, const int table[],
                 int outputBits, int inputBits) {
  uint64_t result = 0;
  for (int i = 0; i < outputBits; i++) {
    result = (result << 1) | ((input >> (inputBits - table[i])) & 1);
  }
  return result;
}

vector<uint64_t> generate_keys(uint64_t key) {
  uint64_t reduced = permute(key, pc1, 56, 64);
  uint32_t left = reduced >> 28;
  uint32_t right = reduced & 0x0FFFFFFF;
  vector<uint64_t> keys;
  for (int shift : shifts) {
    left = ((left << shift) | (left >> (28 - shift))) & 0x0FFFFFFF;
    right = ((right << shift) | (right >> (28 - shift))) & 0x0FFFFFFF;
    keys.push_back(permute((uint64_t(left) << 28) | right, pc2, 48, 56));
  }
  return keys;
}

uint32_t round_function(uint32_t right, uint64_t key) {
  uint64_t expanded = permute(right, expansion, 48, 32) ^ key;
  uint32_t result = 0;
  for (int i = 0; i < 8; i++) {
    int value = (expanded >> (42 - 6 * i)) & 63;
    int row = ((value & 32) >> 4) | (value & 1);
    int column = (value >> 1) & 15;
    result = (result << 4) | sbox[i][row][column];
  }
  return permute(result, permutation, 32, 32);
}

uint64_t des_cipher(uint64_t block, uint64_t key, bool decrypt) {
  vector<uint64_t> keys = generate_keys(key);
  uint64_t initial = permute(block, initialPermutation, 64, 64);
  uint32_t left = initial >> 32;
  uint32_t right = initial & 0xFFFFFFFF;
  for (int i = 0; i < 16; i++) {
    uint32_t next = left ^ round_function(right, keys[decrypt ? 15 - i : i]);
    left = right;
    right = next;
  }
  return permute((uint64_t(right) << 32) | left, finalPermutation, 64, 64);
}

bool read_hex(string prompt, uint64_t &value) {
  string input;
  cout << prompt;
  if (!(cin >> input) || input.size() != 16) return false;
  value = 0;
  for (char digit : input) {
    int number;
    if (digit >= '0' && digit <= '9') number = digit - '0';
    else if (digit >= 'A' && digit <= 'F') number = digit - 'A' + 10;
    else if (digit >= 'a' && digit <= 'f') number = digit - 'a' + 10;
    else return false;
```

```
    value = (value << 4) | number;
  }
  return true;
}

uint64_t triple_des(uint64_t block, uint64_t key1,
                    uint64_t key2, uint64_t key3, bool decrypt) {
  if (!decrypt) {
    block = des_cipher(block, key1, false);
    block = des_cipher(block, key2, true);
    return des_cipher(block, key3, false);
  }
  block = des_cipher(block, key3, true);
  block = des_cipher(block, key2, false);
  return des_cipher(block, key1, true);
}

int main() {
  uint64_t block, key1, key2, key3;
  if (!read_hex("Enter the plaintext (16 hex digits): ", block) ||
      !read_hex("Enter key 1 (16 hex digits): ", key1) ||
      !read_hex("Enter key 2 (16 hex digits): ", key2) ||
      !read_hex("Enter key 3 (16 hex digits): ", key3)) {
    cout << "Each input must contain exactly 16 hex digits.\n";
    return 1;
  }
  uint64_t encrypted = triple_des(block, key1, key2, key3, false);
  cout << hex << uppercase << setfill('0');
  cout << "Encrypted: " << setw(16) << encrypted << endl;
  cout << "Decrypted: " << setw(16)
       << triple_des(encrypted, key1, key2, key3, true) << endl;
  return 0;
}
```

Captured Output

```
$ ./build/triple-des
Enter the plaintext (16 hex digits): 0123456789ABCDEF
Enter key 1 (16 hex digits): 0123456789ABCDEF
Enter key 2 (16 hex digits): 23456789ABCDEF01
Enter key 3 (16 hex digits): 456789ABCDEF0123
Encrypted: F2AFD84EE809E2B5
Decrypted: 0123456789ABCDEF
```

EXPERIMENT 7 **Date: 24.08.2026**

Advanced Encryption Standard (AES)

Question

Write a C++ program to implement AES-128 encryption and decryption of one 128-bit block. Include key expansion, SubBytes, ShiftRows, MixColumns, AddRoundKey and their inverse operations.

Enter exactly 32 hexadecimal digits for both plaintext and key. The state is stored column-wise, and the final round omits MixColumns. No padding is added.

C++ Program

aes.cpp

```
#include <iostream>
#include <iomanip>
#include <sstream>
#include <string>
#include <vector>
using namespace std;

int multiply(int a, int b) {
  int result = 0;
  while (b > 0) {
    if (b & 1) result ^= a;
    a <<= 1;
    if (a & 0x100) a ^= 0x11B;
    b >>= 1;
  }
  return result;
}

int field_power(int base, int exponent) {
  int result = 1;
  while (exponent > 0) {
    if (exponent & 1) result = multiply(result, base);
    base = multiply(base, base);
    exponent >>= 1;
  }
  return result;
}

int rotate_byte(int value, int amount) {
  return ((value << amount) | (value >> (8 - amount))) & 255;
}

void generate_sboxes(vector<int> &sbox, vector<int> &inverse) {
  for (int i = 0; i < 256; i++) {
    int value = i == 0 ? 0 : field_power(i, 254);
    int substituted = value ^ rotate_byte(value, 1) ^
                      rotate_byte(value, 2) ^ rotate_byte(value, 3) ^
                      rotate_byte(value, 4) ^ 0x63;
    sbox[i] = substituted;
    inverse[substituted] = i;
  }
}

vector<int> expand_key(vector<int> key, const vector<int> &sbox) {
  int rcon = 1;
  while (key.size() < 176) {
    int size = key.size();
    vector<int> temp(key.end() - 4, key.end());
    if (size % 16 == 0) {
      int first = temp[0];
      for (int i = 0; i < 3; i++) temp[i] = sbox[temp[i + 1]];
      temp[3] = sbox[first];
      temp[0] ^= rcon;
      rcon = multiply(rcon, 2);
    }
    for (int i = 0; i < 4; i++) key.push_back(key[size - 16 + i] ^ temp[i]);
  }
  return key;
}
```

```
void add_round_key(vector<int> &state, const vector<int> &keys, int round) {
  for (int i = 0; i < 16; i++) state[i] ^= keys[16 * round + i];
}

void substitute(vector<int> &state, const vector<int> &table) {
  for (int &value : state) value = table[value];
}

void shift_rows(vector<int> &state, bool inverse) {
  vector<int> previous = state;
  for (int row = 0; row < 4; row++) {
    for (int column = 0; column < 4; column++) {
      int source = (column + (inverse ? 4 - row : row)) % 4;
      state[4 * column + row] = previous[4 * source + row];
    }
  }
}

void mix_columns(vector<int> &state, bool inverse) {
  int forward[4][4] = {{2,3,1,1}, {1,2,3,1}, {1,1,2,3}, {3,1,1,2}};
  int backward[4][4] = {
    {14,11,13,9}, {9,14,11,13}, {13,9,14,11}, {11,13,9,14}
  };
  vector<int> previous = state;
  for (int column = 0; column < 4; column++) {
    for (int row = 0; row < 4; row++) {
      int result = 0;
      for (int i = 0; i < 4; i++) {
        int factor = inverse ? backward[row][i] : forward[row][i];
        result ^= multiply(factor, previous[4 * column + i]);
      }
      state[4 * column + row] = result;
    }
  }
}

vector<int> aes_cipher(vector<int> state, const vector<int> &keys,
                       const vector<int> &sbox, const vector<int> &inverse,
                       bool decrypt) {
  if (!decrypt) {
    add_round_key(state, keys, 0);
    for (int round = 1; round <= 10; round++) {
      substitute(state, sbox);
      shift_rows(state, false);
      if (round != 10) mix_columns(state, false);
      add_round_key(state, keys, round);
    }
  } else {
    add_round_key(state, keys, 10);
    for (int round = 9; round >= 0; round--) {
      shift_rows(state, true);
      substitute(state, inverse);
      add_round_key(state, keys, round);
      if (round != 0) mix_columns(state, true);
    }
  }
  return state;
}

bool read_hex(string prompt, vector<int> &bytes) {
  string input;
  cout << prompt;
  if (!(cin >> input) || input.size() != 32) return false;
  bytes.clear();
  for (int i = 0; i < 32; i += 2) {
    int value = 0;
    for (int j = 0; j < 2; j++) {
      char digit = input[i + j];
      int number;
      if (digit >= '0' && digit <= '9') number = digit - '0';
      else if (digit >= 'a' && digit <= 'f') number = digit - 'a' + 10;
      else if (digit >= 'A' && digit <= 'F') number = digit - 'A' + 10;
      else return false;
      value = value * 16 + number;
```

```
    }
    bytes.push_back(value);
  }
  return true;
}

string to_hex(const vector<int> &bytes) {
  ostringstream output;
  output << hex << uppercase << setfill('0');
  for (int value : bytes) output << setw(2) << value;
  return output.str();
}

int main() {
  vector<int> block, key, sbox(256), inverse(256);
  if (!read_hex("Enter the plaintext (32 hex digits): ", block) ||
      !read_hex("Enter the key (32 hex digits): ", key)) {
    cout << "Each input must contain exactly 32 hex digits.\n";
    return 1;
  }
  generate_sboxes(sbox, inverse);
  vector<int> keys = expand_key(key, sbox);
  vector<int> encrypted = aes_cipher(block, keys, sbox, inverse, false);
  cout << "Encrypted: " << to_hex(encrypted) << endl;
  cout << "Decrypted: "
       << to_hex(aes_cipher(encrypted, keys, sbox, inverse, true)) << endl;
  return 0;
}
```

Captured Output

```
$ ./build/aes
Enter the plaintext (32 hex digits): 00112233445566778899AABBCCDDEEFF
Enter the key (32 hex digits): 000102030405060708090A0B0C0D0E0F
Encrypted: 69C4E0D86A7B0430D8CDB78070B4C55A
Decrypted: 00112233445566778899AABBCCDDEEFF
```

EXPERIMENT 8 **Date: 05.10.2026**

Miller-Rabin Primality Test

Question

Write a C++ program to test whether a given integer is prime using the Miller-Rabin primality test. Handle small and even integers, accept a positive number of rounds and choose a random base for each round.

Write $n-1=2^s d$ with odd d. An odd candidate passes a round if $a^d \equiv 1 \pmod{n}$ or $a^{2^r d} \equiv -1 \pmod{n}$ for some $0 \leq r < s$. A passing candidate is reported as probably prime.

C++ Program

`miller-rabin.cpp`

```cpp
#include <iostream>
#include <random>
using namespace std;

long long generate_random_test_number(long long limit) {
  static random_device rd;
  static mt19937_64 gen(rd());
  uniform_int_distribution<long long> distribution(2, limit);
  return distribution(gen);
}

long long multiply_mod(long long a, long long b, long long modulus) {
  return (unsigned __int128)a * b % modulus;
}

long long fermats_primality_test(long long exponent, long long base,
                                 long long prime) {
  if (exponent == 0) return 1;
  if (exponent % 2 == 0) {
    return fermats_primality_test(exponent / 2,
                                  multiply_mod(base, base, prime), prime);
  }
  return multiply_mod(base,
                      fermats_primality_test(exponent - 1, base, prime),
                      prime);
}

bool test_sequence(long long prime, long long base) {
  long long d = prime - 1;
  int s = 0;
  while (d % 2 == 0) {
    d /= 2;
    s++;
  }
  long long result = fermats_primality_test(d, base, prime);
  if (result == 1 || result == prime - 1) return true;
  for (int i = 1; i < s; i++) {
    result = multiply_mod(result, result, prime);
    if (result == prime - 1) return true;
    if (result == 1) return false;
  }
  return false;
}

int main() {
  cout << "Enter the prime number you want to do this test for: ";
  long long input;
  if (!(cin >> input)) return 1;
  if (input < 2) {
    cout << input << " is not prime" << endl;
    return 0;
  }
  if (input == 2 || input == 3) {
    cout << input << " is prime" << endl;
    return 0;
  }
  if (input % 2 == 0) {
    cout << input << " is composite (even)" << endl;
    return 0;
```

```
  }
  cout << "Enter the number of rounds you want to perform: ";
  int rounds;
  if (!(cin >> rounds) || rounds < 1) {
    cout << "The number of rounds must be positive.\n";
    return 1;
  }
  for (int i = 0; i < rounds; i++) {
    long long test = generate_random_test_number(input - 2);
    if (!test_sequence(input, test)) {
      cout << input << " is composite (witness: " << test
           << ", round " << i + 1 << ")" << endl;
      return 0;
    }
  }
  cout << input << " is probably prime after " << rounds << " rounds"
       << endl;
  return 0;
}
```

Captured Output

```
$ ./build/miller-rabin
Enter the prime number you want to do this test for: 97
Enter the number of rounds you want to perform: 8
97 is probably prime after 8 rounds

$ ./build/miller-rabin
Enter the prime number you want to do this test for: 100
100 is composite (even)
```

EXPERIMENT 9 **Date: 12.10.2026**

RSA Encryption

Question

Write a C++ program to generate a textbook RSA public/private key pair from distinct primes p and q and a valid public exponent e, then encrypt and decrypt an integer message m satisfying $0 \leq m < n$.

Use $n = pq$, $\phi(n) = (p-1)(q-1)$, $d = e^{-1} \bmod \phi(n)$, $c = m^e \bmod n$ and $m = c^d \bmod n$. The program accepts primes at most 1,000,000 and uses no message padding.

C++ Program

rsa.cpp

```
#include <iostream>
#include <numeric>
using namespace std;

bool is_prime(long long number) {
  if (number < 2) return false;
  for (long long i = 2; i * i <= number; i++) {
    if (number % i == 0) return false;
  }
  return true;
}

long long modular_power(long long base, long long exponent,
                        long long modulus) {
  if (exponent == 0) return 1 % modulus;
  if (exponent % 2 == 0) {
    long long square = (unsigned __int128)base * base % modulus;
    return modular_power(square, exponent / 2, modulus);
  }
  return (unsigned __int128)base *
         modular_power(base, exponent - 1, modulus) % modulus;
}

long long extended_gcd(long long a, long long b,
                       long long &x, long long &y) {
  if (b == 0) {
    x = 1;
    y = 0;
    return a;
  }
  long long nextX, nextY;
  long long result = extended_gcd(b, a % b, nextX, nextY);
  x = nextY;
  y = nextX - (a / b) * nextY;
  return result;
}

int main() {
  long long p, q, e, message;
  cout << "Enter two distinct primes p and q (2..1000000): ";
  if (!(cin >> p >> q) || p < 2 || q < 2 ||
      p > 1000000 || q > 1000000 || p == q ||
      !is_prime(p) || !is_prime(q)) {
    cout << "Invalid prime numbers.\n";
    return 1;
  }
  long long n = p * q;
  long long phi = (p - 1) * (q - 1);
  cout << "Enter the public exponent e: ";
  if (!(cin >> e) || e <= 1 || e >= phi || gcd(e, phi) != 1) {
    cout << "Use 1 < e < phi(n) with gcd(e, phi(n)) = 1.\n";
    return 1;
  }
  long long x, y;
  extended_gcd(e, phi, x, y);
  long long d = (x % phi + phi) % phi;
  cout << "Enter the integer message (0 <= m < " << n << "): ";
  if (!(cin >> message) || message < 0 || message >= n) {
    cout << "The message must belong to 0..n-1.\n";
```

```
    return 1;
  }
  long long encrypted = modular_power(message, e, n);
  cout << "n = " << n << ", phi(n) = " << phi << endl;
  cout << "Public key (e, n): (" << e << ", " << n << ")" << endl;
  cout << "Private key (d, n): (" << d << ", " << n << ")" << endl;
  cout << "Encrypted: " << encrypted << endl;
  cout << "Decrypted: " << modular_power(encrypted, d, n) << endl;
  return 0;
}
```

Captured Output

```
$ ./build/rsa
Enter two distinct primes p and q (2..1000000): 61 53
Enter the public exponent e: 17
Enter the integer message (0 <= m < 3233): 65
n = 3233, phi(n) = 3120
Public key (e, n): (17, 3233)
Private key (d, n): (2753, 3233)
Encrypted: 2790
Decrypted: 65
```

Verification and References

Verification. All 11 programs compiled with C++17, `-Wall` and `-Wextra` without warnings. The verification script completed 16,962 checks, including:

- Known-answer tests for the classical ciphers, DES, AES-128 and RSA.
- 75 randomized DES, three-key Triple DES and AES-128 ciphertext comparisons against OpenSSL, plus decryption back to each original block.
- Miller-Rabin modular exponentiation and witness sequences checked against Python arbitrary-precision arithmetic, including large signed 64-bit inputs.
- Odd integers from 5 through 2,999 classified against trial division with fixed test bases; small, even and invalid input cases checked separately.
- Invalid Hill keys, malformed hex, invalid OTP key lengths and invalid RSA parameters rejected.

Reproduce the checks and outputs.

```
python3 verify.py
python3 capture_outputs.py
```

The verification script requires Python 3.10+, GCC and OpenSSL 3 with the default and legacy providers. Output capture uses a POSIX pseudo-terminal. The archived output files preserve the runs printed in this record; Miller-Rabin witnesses can vary on later composite-number runs.

Repository.

`https://github.com/mohnishc07/Modern-Cryptography`

Algorithm references.

1. National Institute of Standards and Technology. *FIPS 46-3: Data Encryption Standard (DES)*. DES permutations, S-boxes, key schedule and Triple DES EDE construction. Historical specification.

   `https://csrc.nist.gov/pubs/fips/46-3/final`

2. National Institute of Standards and Technology. *FIPS 197: Advanced Encryption Standard (AES)*. AES round operations, field arithmetic, key expansion and example test values.

   `https://csrc.nist.gov/pubs/fips/197/final`

3. A. Menezes, P. van Oorschot and S. Vanstone. *Handbook of Applied Cryptography*, Chapter 4, Algorithm 4.24: Miller-Rabin probabilistic primality test.

   `https://cacr.uwaterloo.ca/hac/about/chap4.pdf`